混合检索的融合与候选裁剪

摘要：本文讨论把稠密向量检索、词法检索与稀疏向量三路结果融合为统一排序的工程做法，以及为控制延迟而做的候选裁剪。结论是融合的收益主要来自通道互补，而延迟的收益几乎全部来自“在重排之前把候选变小”。

1 引言。Any single retrieval channel has a characteristic failure mode: dense retrieval misses rare lexical anchors, while lexical retrieval misses paraphrases. 融合的价值就是把两种失败模式互相补上，因此融合前要先确认两路真的在失败模式上互补。

2 词法通道的中文问题。词法检索依赖分词质量，字符级切分会让中文关键词的统计信号严重稀释。我们采用的方案是按词切分，实际效果是 word-level segmentation is what makes BM25 usable for Chinese text。分词器缺失时应回退到字符级而不是直接失败，否则一个可选依赖会变成硬故障点。

3 融合算法。我们使用倒数排名融合，它对通道分数不可比这件事天然免疫，因为只用排名。融合常数的物理含义是平滑程度，融合常数越大则排在后面的文档被压得越平缓。经验取值在五十到七十之间；过大则融合退化为原始排名的简单平均，过小则头部排名过度主导结果。

4 候选裁剪的必要性。candidate pruning must happen before reranking because reranking dominates the cost。在本地单用户设备上我们实测重排占单次检索总耗时的九成以上，因此把候选数从五十降到二十，可以在前五名结果完全一致的前提下把总耗时下降约六成。裁剪的依据是融合分数，而不是各通道的原始分数。

5 重排的成本结构。交叉编码器要为每个候选单独跑一次前向因此单候选成本是固定开销。这意味着重排耗时与候选数近似线性，而每个候选的耗时取决于它的长度：注意力计算要对整段文本做一遍，所以把明显过长的候选截断，是仅次于减小候选数的第二个杠杆。

表 1 候选数与延迟。候选二十 重排约 650 毫秒；候选五十 重排约 1490 毫秒；截断至三百八十四 token 后再降约六成；半精度推理再降约四成。

6 关于分数归一化。分数归一化只解决量纲问题不解决通道质量问题。把各通道分数映射到同一区间只能保证量纲一致，当某个通道整体质量下降时，归一化后的分数依然会把噪声推到前列；要发现这种情况必须看逐通道的命中率，而不是看融合后的总分。

7 查询扩展。multi-query rewriting widens recall at the price of more reranking work。改写得到的多个查询并行检索再合并，能显著提升长尾问题的召回，但候选数随之膨胀，重排成本成倍上升；因此查询扩展必须与候选裁剪配套使用，否则用户感知到的是更慢而不是更准。

8 失败语义。检索服务不可用与检索结果为空是两件事：前者是系统故障，应提示用户稍后重试；后者表示文档中确实没有相关内容，应提示用户换一种问法或补充文献。把两者混为一谈会让用户在系统故障时误以为需要重新上传文档。

10 索引与增量更新。新文档入库后，最容易被忽略的是缓存一致性：检索缓存按查询键索引，而查询结果依赖整个语料集合，增量更新必须让整个索引的缓存同时失效。做不到这一点时，用户会看到“刚上传的文档检索不到”这类看起来像索引失败的现象。

11 成本模型。三个阶段的伸缩行为完全不同：the three stages have very different scaling behaviour。编码成本只在文档入库时发生，与查询无关；召回成本与语料规模近似对数增长；而重排成本与候选数线性相关，因此它是唯一一个会随调参而剧烈变化的阶段。

12 候选去重。多查询与多通道合并后会出现重复候选，重复项会挤占重排预算。去重应当以块标

识为准而不是以文本内容为准，因为相同内容出现在不同文档里属于两处独立证据，按文本去重会丢掉其中一处。

１３　并行执行。召回阶段的三条通道彼此独立，可以并行；重排阶段是单次批推理，并行只会争抢同一块显存。因此并行化的收益集中在召回阶段，把重排也拆成多进程在本地单卡设备上只会更慢。

１４　运行时开关。回退开关必须是运行时可切换的而不是编译期常量。把档位做成编译期常量会让线上回退变成一次重新构建与重启，而质量异常往往需要分钟级的回退能力。开关变更后必须连带清理已缓存的检索组件，否则新档位不会生效，表现为“改了配置但结果一模一样”。

１５　监控指标。延迟与总体命中率都不足以发现单通道退化。我们的结论是：逐通道命中率是唯一能发现某个通道整体退化的指标。具体做法是分别记录三条通道各自贡献的候选进入前五名的比例，当某个通道的贡献突然趋近于零时，说明它已经失效而不是“这题太难”。

１６　评测方法。比较两种融合配置时应当做双臂对照：同一份用例集、同一台机器、同一组候选参数，只改被考察的那一项。若同时改动多项，即便指标变好也无法归因，后续会在别的场景里遭遇反弹。

１７　常见故障。检索为空、检索超时、结果与预期文档不符，这三类现象的根因完全不同：空结果通常是分词或阈值问题，超时通常是候选过大，结果不符则多是跨文档混淆。把它们都当作“检索不准”处理，会浪费大量排查时间。

１９　候选池与指标口径。候选池的大小直接决定精确率的分母。当候选只有两三个时，精确率会在很小的分母上取值，看起来很高却没有意义。因此比较两组参数之前，先要确认候选池的规模与线上一致。

２０　单篇与整库的差别。只挂一篇文档时检索指标会系统性地偏乐观。原因很直接：相关文档的数量固定，而候选总数被压得很小，于是模型几乎不可能把证据挤出前几名。这类失真不会报错，只会让指标偏好看。

２１　评测与线上的对齐。把整库检索换成单篇检索是评测里最常见的口径失真。线上检索永远面对整个文献库，因此评测也应当让检索看到足够多的干扰文档，否则测的是「在小池塘里捞鱼」而不是真实难度。

２２　干扰文档的选择。干扰文档不是随便挂几篇就够，同主题、术语重合度高的文档才构成真实干扰。若干扰文档与问题毫无关系，检索系统只要抓住一个关键词就能分清，评测难度会被严重低估。

２３　融合常数的敏感性。融合常数在经验区间内的小幅变化通常不会改变前几名结果，因此调它之前应当先确认指标对该参数确实敏感；若扫描一圈指标几乎不动，说明瓶颈不在这里，继续调只是浪费时间。

２４　通道权重的误区。三路召回等权融合是默认值而不是最优值。当某一路在特定语料上整体偏弱时，等权会持续把它不合格的候选推入前列，而这类问题在总体指标上往往被其它两路掩盖。

２５　查询长度的影响。短查询主要依赖词法通道，长查询主要依赖语义通道。因此按查询长度分层看指标，比看一个总体均值更能定位问题：总体持平但短查询变差、长查询变好，是典型的通道此消彼长。

２６　失败注入。为验证评测的敏感性，可以故意注入一个已知的退化，例如把候选数砍半或关掉一路召回，看指标是否明显下降。若注入退化后指标不动，说明这套评测对真实退化不敏感，此时应当先修评测而不是修系统。

２７　观测与告警。线上应当持续记录逐通道命中率与检索耗时，并对明显偏离基线的时段告警。评测是低频的离线动作，它无法替代线上观测——两者一个回答“行不行”，一个回答“现在有没有变坏”。

２９　用例集的维护。检索用例会随语料变化而逐渐失效：被删掉的文档会让　ｇｏｌｄ　指向不存在的块，改了标题会让同义词失效。因此用例集需要一个最小的自检：每次运行前确认每条　ｇｏｌｄ　都能定位到真实块，定位失败就整体报错而不是把该条悄悄算成未命中。

３０　干扰文档的规模。干扰文档太少时，检索只要抓住一个关键词就能命中；干扰文档太多又会让评测成本急剧上升。经验做法是让候选总数远大于截断位次，这样前几名的排序才有区分度。

３１　结论。混合检索的调优顺序应当是：先确认通道互补，再调融合常数，最后在重排之前裁剪候选。反过来做会得到调不动、说不清、也复现不了的结果。